

LUCRAREA 3

LUCRUL ÎN ÎNTRERUPERI CU MICROPROCESORUL Z - 80

1. OBIECTUL LUCRĂRII

Lucrarea își propune studierea și însușirea noțiunilor teoretice precum și rezolvarea unor probleme concrete legate de lucrul în întreruperi cu circuitele I/O din familia microprocesorului Z-80.

2. BREVIAR TEORETIC

În scrierea programelor apar situații în care un program conține mai multe părți ce necesită o execuție simultană. Programarea acestor tipuri de aplicații este o **programare paralelă sau concurentă**. Orice aplicație în timp real conține de obicei astfel de operații ce se derulează paralel. Un program ce se execută în timp real trebuie să răspundă practic instantaneu la semnalele ce se aplică din exterior, iar “timpul real” în care sistemul de calcul este ocupat cu o anumită operație trebuie să fie strâns corelat cu timpul în care se produc alte evenimente, în vederea tratării tuturor acestora. Un program ce realizează coordonarea mai multor activități în același timp se cheamă cu facilități de tip **multitasking**. Lucrul în întreruperi prezintă avantajul faptului că permite procesorului gestionarea mai multor activități, practic simultan. Sunt mai multe tehnici prin care acest lucru este posibil.

O tehnică implică gestionarea unor activități în programul principal, iar apariția altor evenimente (activități sau task-uri) ce necesită tratare, este semnalizată procesorului prin cereri de întrerupere. În aceste situații, procesorul suspendă activitățile în curs din programul principal și trece la tratarea evenimentelor ce au survenit. După tratarea acestora, revine în programul principal în locul în care a apărut suspendarea. Este posibilă chiar suspendarea tratării unei cereri de întrerupere în situația apariției unei a doua apărute cu grad crescut de prioritate (întreruperi imbricate).

O a doua tehnică presupune alocarea unui timp de tratare bine precizat (egal sau diferit) fiecărei activități ce trebuie gestionată (**time sharing** sau **time slicing**). Împărțirea timpului alocat fiecărei activități se poate face prin folosirea unui circuit Z-80 CTC, programat corespunzător, care va atenționa procesorul prin cereri de întrerupere la sfârșitul perioadei alocate tratării task-ului respectiv.

În ambele situații, viteza mare de execuție a procesorului lasă impresia că task-urile se execută simultan, ca și când fiecare task deține controlul. În acest caz se spune că are loc o execuție pseudo-paralelă sau execuție paralelă logică.

Planificarea spre execuție a task-urilor se poate face în două moduri:

- în procedura “round-robin” prin care task-urile așezate într-o coadă de așteptare sunt executate succesiv fie până la terminare normală, fie un interval de timp bine determinat; un task după ce a fost executat este trecut într-o coadă de așteptare și este lansat task-ul următor;
- în procedura “după priorități” prin care fiecărui task i se atribuie o prioritate, iar dintre task-urile ce concurează pentru controlul procesorului, se va trece în execuție pe cel având prioritatea cea mai mare.

În cadrul oricărui program cu execuție paralelă apar două categorii de task-uri: disjuncte (independente) sau care interacționează (cooperează). Două sau mai multe task-uri se cheamă disjuncte dacă nu schimbă informații între ele sau dacă nu utilizează resurse ale microsistemului în comun. În caz contrar se spune că ele interacționează.

Interacțiunea dintre task-uri poate fi redusă la trei tipuri de operații:

- comunicarea, ce permite schimbul de date între task-uri;
- sincronizarea, prin care se asigură eșalonarea corectă a execuției acestora;
- excluderea mutuală, prin care se realizează utilizarea corectă a resurselor comune (excluderea accesului simultan a mai multor task-uri la aceeași resursă).

Pentru rezolvarea acestor operații se pot folosi mai multe procedee, dintre care amintim:

- folosirea unor variabile (pe bit sau octet) de tip “semafor”, prin care sunt semnalizate evenimente ce apar într-un task în vederea tratării într-un altul;
- folosirea unor structuri de date denumite “mesaje” și a unei zone de memorie în care se depune mesajul, denumit “celula mesaj” sau “cutie poștală”, prin care un task semnalizează și oferă date altuia.

Microprocesorul Z-80 are două mecanisme de întreruperi prin care se ușurează scrierea de aplicații de tip “multitasking” de către un utilizator (întreruperi nemascabile și, respectiv, mascabile). Întreruperile nemascabile au caracter prioritar fiind acceptate în orice condiții de către procesor. Tratarea acestora se face prin apelarea unei subrutine de tratare, pe care utilizatorul trebuie să o scrie începând cu adresa rezervată de memorie 0066h. Subrutina de tratare trebuie să se termine cu instrucțiunea de revenire RETN. Întreruperile mascabile sunt acceptate de procesor doar dacă mecanismul de întreruperi mascabil al acestuia este validat (prin rularea unei instrucțiuni de tip EI). Sunt trei posibilități de lucru cu întreruperile mascabile:

- modul 0 (analog procesorului 8080, în care circuitul I/O ce a solicitat întrerupere indică ce se va executa odată cu întreruperea tratării propriu-zise a întreruperii - de obicei o instrucțiune tip CALL n16, adică apelul unei subrutine de tratare);
- modul 1 (ce implică o adresă unică a subrutinei de tratare, adresa 0038h). Circuitul ce a solicitat întrerupere nu trebuie să ofere o altă informație procesorului;
- modul 2 (modul cel mai complex, care va fi explicat în cele ce urmează).

Modul 2 de tratare a întreruperilor mascabile folosit de μP Z80

Fiecare circuit periferic își are propriul vector de întrerupere. Registrul I al procesorului rămâne nemodificat în timpul întreruperilor generate în sistem. Astfel, toate cererile de întreruperi pot fi concentrate din punct de vedere al precizării adreselor subrutinelor de tratare, într-o tabelă unică numită *tabela vectorilor de întrerupere*. O întrerupere dispune în acest tabel de două locații în care se scriu ocmps-ul urmat de ocms-ul adresei de memorie unde se va trata întreruperea respectivă (adresa subrutinei de tratare).

Sarcina programatorului la lucrul cu întreruperi constă în scrierea registrului I al μP -ului, scrierea registrului V de la circuitul periferic ce va solicita întreruperea, completarea corectă a tabelii vectorilor de întrerupere cu adresele subrutinelor de tratare și scrierea propriu-zisă a subrutinei de tratare. La apariția unei întreruperi, după acceptarea acesteia de către microprocesor (lucru indicat prin activarea simultană a semnalelor M1 și IORQ), circuitul I/O ce a solicitat întreruperea va depune pe magistrala de date vectorul de întrerupere, în vederea

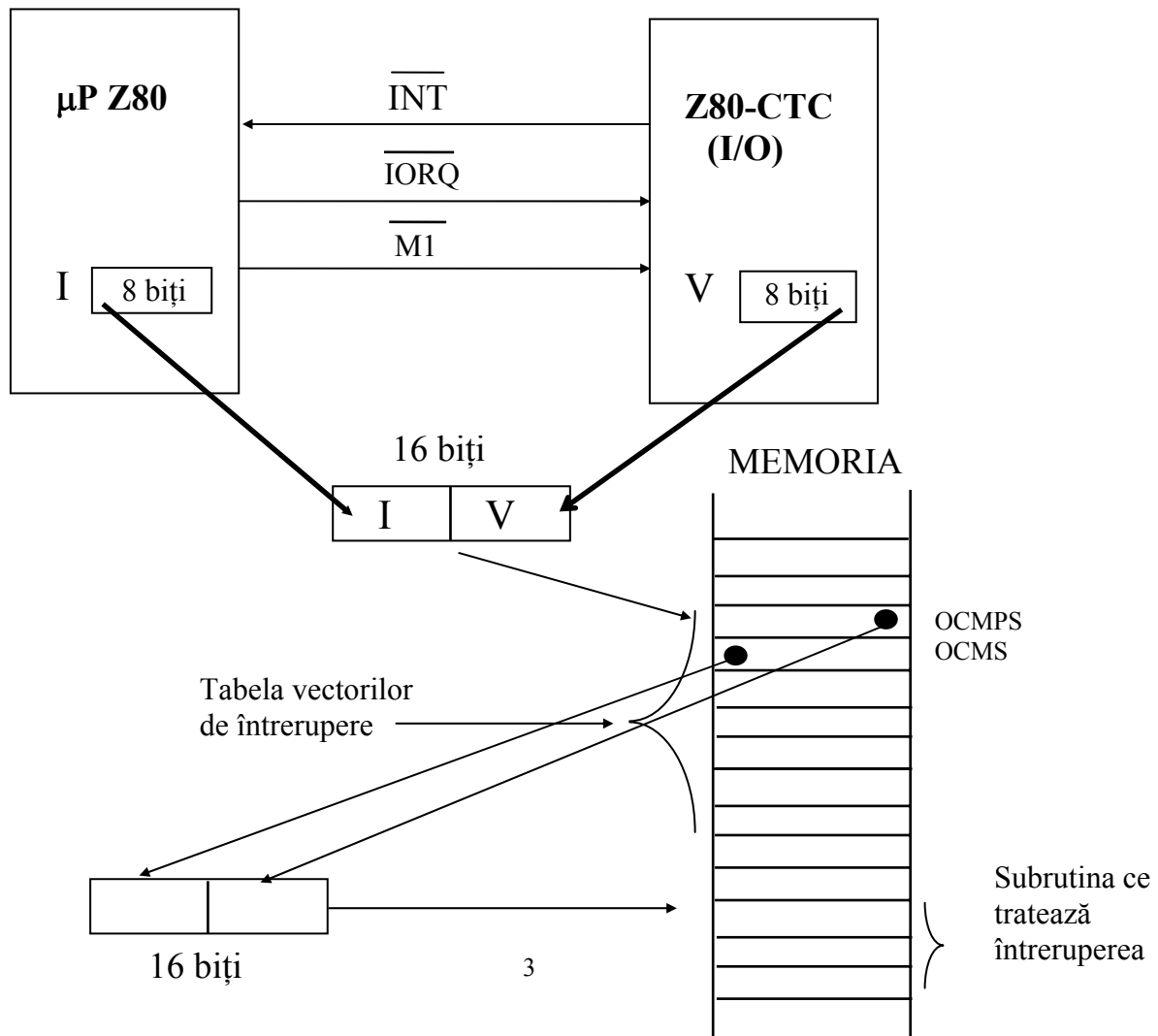
preluării acestuia de către procesor. Structura hardware a circuitului I/O nu se complică mult, iar subrutina de tratare se poate plasa oriunde în memorie.

Lucrul cu întreruperile mascabile la microprocesorul Z-80

Prin lucrul în întreruperi se obțin următoarele avantaje în scrierea programelor:

- se permite procesorului să gestioneze mai multe activități, deși la un moment dat nu se poate ocupa decât de una singură. După cum s-a mai aratat, există mai multe tehnici prin care acest lucru se poate realiza. Una dintre ele presupune împărțirea timpului de lucru al procesorului în intervale egale sau inegale, gestiunea fiecărei activități realizându-se într-un astfel de interval (time-sharing). Dacă durata intervalului este suficient de mică și dacă procesul este repetitiv, senzația pentru un observator extern este aceea că procesorul gestionează practic simultan respectivele activități (multitasking). Pentru tehnica prezentată, împărțirea timpului de lucru, se poate realiza prin întreruperi generate de un circuit de tip CTC, programat corespunzător.
- procesorul nu este obligat să urmărească fiecare activitate, ci poate fi înștiințat de apariția unui eveniment în vederea tratării acestuia. Dacă respectivele activități apar rar, lucrul în varianta time-sharing poate să devină un inconvenient, mai ales în situația unui program principal încărcat. Este de preferat în aceasta situație întreruperea pentru scurt timp a activității din programul principal în vederea tratării evenimentului apărut și semnalizat prin cererea de întrerupere.

Schema de tratare a întreruperilor mascabile în modul 2 este următoarea:



La lucrul în întreruperi cu microprocesorul Z-80 (pentru modul 2 de lucru) trebuie urmărite **regulile**:

- bază stivei trebuie inițializată, prin încărcarea registrului SP cu adresa de bază a stivei. Stiva trebuie creată în memoria RAM externă, într-un spațiu care nu poate fi folosit în alte scopuri. Se va avea în vedere o dimensiune a acestui spațiu acoperitoare necesităților programului. Se va avea în vedere faptul că la depunerea în stivă, acesta crește către adresele de început ale memoriei, precum și faptul că mecanismele procesorului accesează stiva în tehnică LIFO (Last In First Out).
- registrul I al microprocesorului trebuie încărcat cu ocms-ul adresei tabelii vectorilor de întrerupere. Se reamintește faptul că singura posibilitate de transfer în registrul I se face prin registrul A.
- precizarea modului de lucru în întreruperi la procesor, prin rularea instrucțiunii IM2.
- programarea pentru lucrul în întreruperi a circuitului I/O care va genera întrerupere. Acest lucru trebuie făcut în contextul general al programării respectivului circuit.
- scrierea registrului vectorului de întrerupere la acest circuit. Vectorul constă practic în ocms-ul adresei din tabela vectorilor de întrerupere, la care se află subrutina de tratare a întreruperii, corespunzătoare respectivului circuit I/O ce a solicitat întrerupere.
- completarea tabelii vectorilor de întrerupere cu adresa subrutinei de tratare. În caz că se lucrează cu mai multe întreruperi, această măsură trebuie luată pentru fiecare situație în parte. Tabela se poate crea atât în zona de memorie ROM, caz în care scrierea este asigurată la punerea la punct a programului, dar și în zona de memorie RAM, situație în care utilizatorul trebuie să ia măsuri pentru înscrierea zonei la reluarea rulării programului.
- scrierea propriu-zisă a subrutinei de tratare a întreruperii. Subrutina trebuie scrisă la adresa declarată în tabela vectorilor de întrerupere. Se vor respecta regulile de scriere a subrutinelor normale. În plus, la subrutinele de tratare ale întreruperilor se va avea în vedere ca acestea să conțină instrucțiunea EI, pentru rearmarea procesorului în vederea primirii unei noi întreruperi (întreruperea sosită și acceptată dezactivând mecanismul de întreruperi mascabile al procesorului). De asemenea, este obligatoriu ca instrucțiunea de revenire din subrutina de tratare să fie una de tip RETI. Aceasta are ca rol rearmarea circuitului I/O care a solicitat întreruperea în vederea generării unei noi întreruperi în viitor.
- în programul principal, se va avea în vedere ca după încheierea înscrierii părții de inițializare, mecanismul de întreruperi mascabil al procesorului să fie validat prin rularea unei instrucțiuni de tip EI. Din acest moment, procesorul va fi apt să primească și să treacă la tratarea întreruperilor mascabile.

Aceste reguli vor trebui să fie împletite armonios cu cerințele ce trebuie rezolvate în respectiva aplicație.

3. PROBLEME REZOLVATE

a. Într-un sistem cu μP Z-80 se află plasate, în spațiul I/O, un circuit Z-80 PIO la adresa de bază 10h ($A0 \equiv \overline{C/D}$; $A1 \equiv \overline{B/A}$) și un circuit Z-80 CTC, la adresa de bază 40h. Pe intrarea CLK/TRG3 de la CTC se aplică un semnal cu frecvența de 100Hz. Realizați un program pentru obținerea pe linia PA0 (PIO) a unui semnal cu frecvența de 10Hz și factor de umplere 2/5 (nu se vor utiliza alte construcții hardware).

Soluție

Cum adresa de bază a circuitului PIO este 10h și, respectiv:

$$\left. \begin{array}{l} A0 = \overline{C/D} \\ A1 = B/A \end{array} \right\} \Rightarrow \begin{array}{l} \text{Adresa de date Port A} = 10h \\ \text{Adresa de comenzi Port A} = 11h \end{array}$$

Portul A se programează în modul 0 (ieșire pe octet).

Cuvântul de comandă: 00001111=0Fh

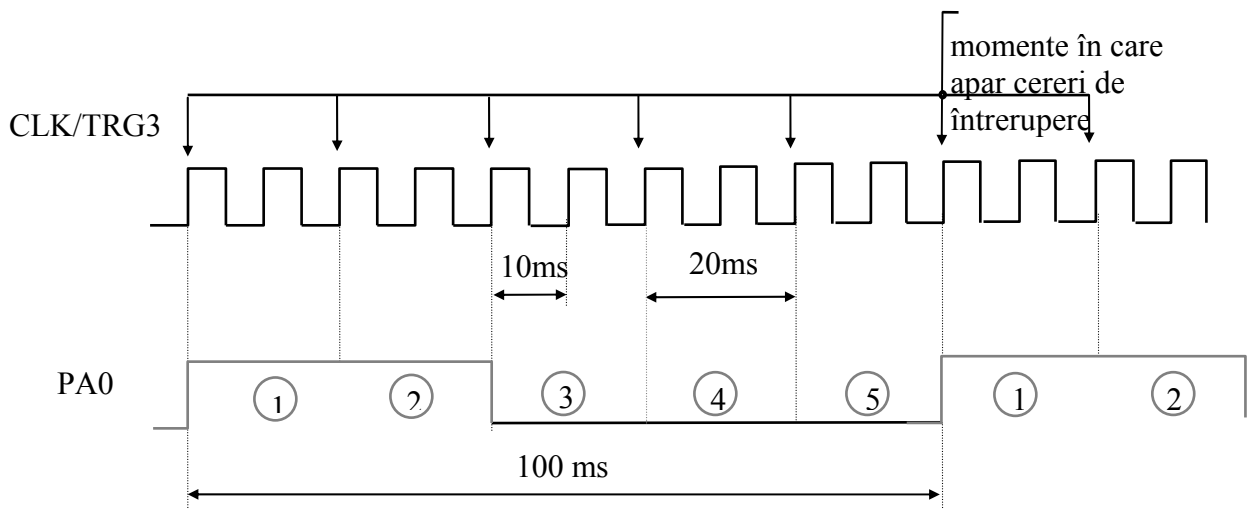
Cum adresa de bază a circuitului CTC este 40h, rezultă că adresa canalului 3 este 43h.

Alegem ca tabela vectorilor de întreruperi să fie plasată în memorie începând cu adresa 200h.

Rezultă că regiștrii I și V trebuie încărcăți cu valorile:

I←02h respectiv, V←00h

Alegem adresa de început a subrutinei de tratare a cererii de întrerupere ca fiind 300h.



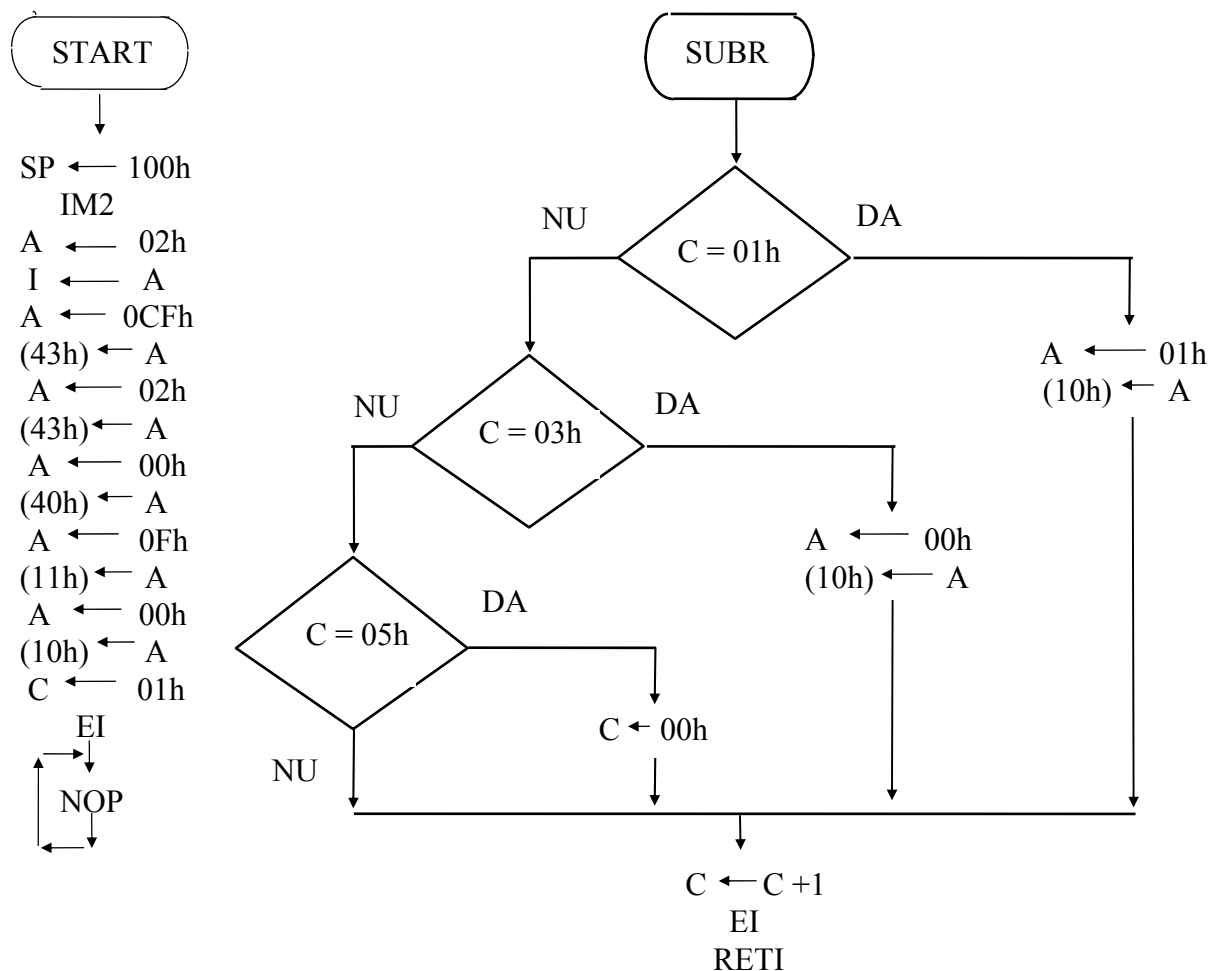
Cuvântul de comandă pentru CTC este $11001111 = 0CFh$.

Constanta de timp este 02h (frecvența de ieșire ZC/TO este jumătate din frecvența CLK/TRG3).

Vectorul de întrerupere $V = 00h$ se trimite pe adresa canalului 0 (40h).

Ideea de rezolvare a problemei implică folosirea unei semnalizări (întreruperi) provenită de la canalul 3 al circuitului Z-80 CTC, ce apare cu o frecvență de 50 Hz sau în timp, la intervale de 20 ms. Aceste semnalizări vor fi numărate, iar semnalul ce se va obține la ieșirea circuitului Z-80 PIO (PA0), va fi menținut în starea "1" logic pe durata a două perioade și, respectiv, în starea "0" logic pe durata a trei perioade. Numărarea semnalizărilor se face în subrutina de tratare a întreruperii provenită de la circuitul Z-80 CTC. Divizarea pe acest canal este realizată cu constanta 2 (pentru a se ajunge la frecvența de 50 Hz). Numărarea se face prin folosirea registrului C al microprocesorului (încărcat la inițializare în programul principal cu valoarea 1). După inițializare, în programul principal nu apare nici o altă sarcină de îndeplinit de către procesor, acesta fiind motivul pentru care se așteaptă într-o buclă infinită sosirea unei întreruperi. După tratarea acesteia se revine iarăși la bucla infinită. Subrutina de tratare a întreruperii se sfârșește cu rularea unei instrucțiuni EI, care rearmează la procesor mecanismul de întreruperi mascabil și, respectiv, printr-o instrucțiune de tip RETI, care marchează sfârșitul logic al subrutinei de tratare, provocând revenirea în programul principal și rearmarea mecanismului de generare a întreruperilor la circuitului Z-80 CTC.

Organigramele programului principal și, respectiv, a subrutinei de tratare a întreruperii de la canalul 3 al circuitului Z-80 CTC sunt următoarele:



În limbaj de asamblare, pentru programul principal se obține:

```
.ORG 0C000h
LD SP, 100h
IM 2
LD A, 02h
LD I, A
LD A, 0CFh
OUT (43h), A
LD A, 02h
OUT (43h), A
LD A, 00h
OUT (40h), A
```

```
LD A, 0Fh
OUT (11h), A
LD A, 00h
OUT (10h), A
LD C, 01h
EI
et1: NOP
JP et1
```

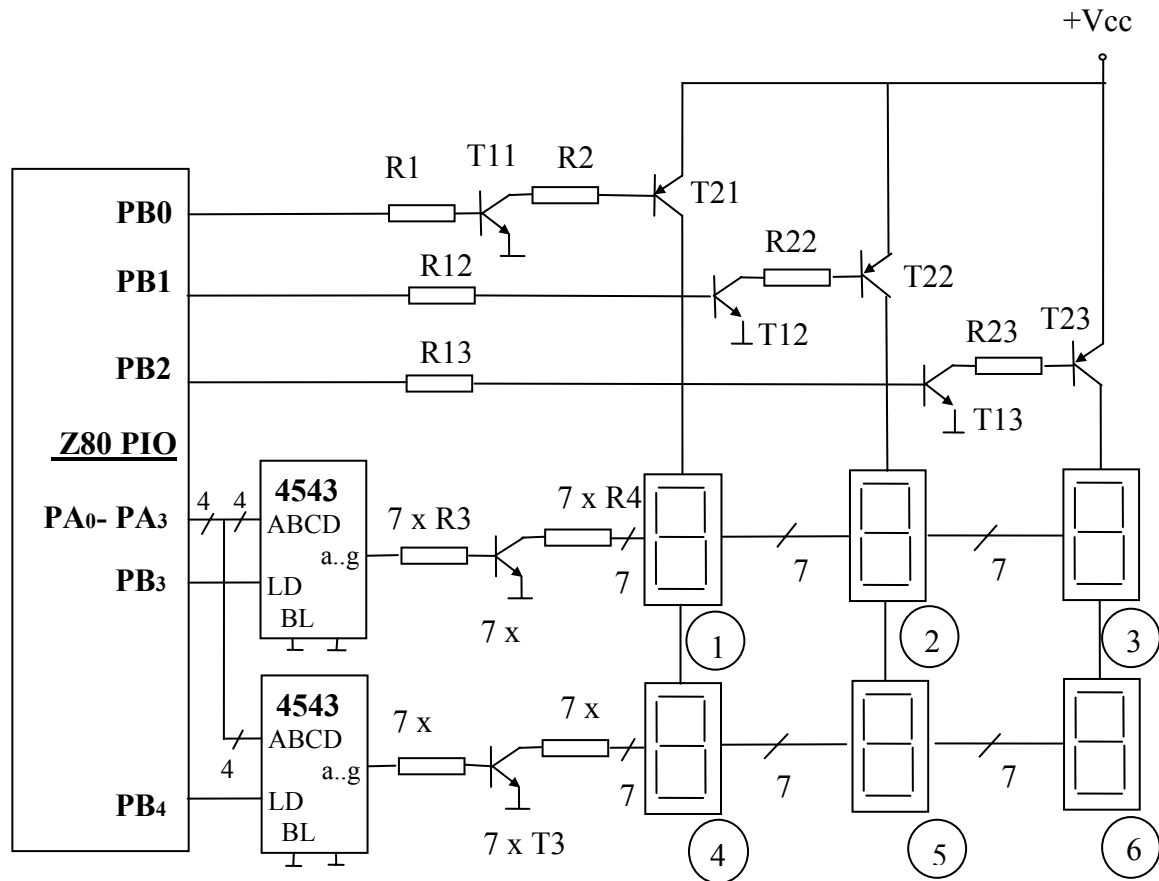
Conținutul tabelii vectorilor de întrerupere:

```
.ORG 0206h
DB 00h, 03h
```

Pentru subrutina de tratare a întreruperii ce provine de la canalul 3 al circuitului Z-80 CTC se obține următorul format obiect:

```
.ORG 0300h
LD A, 01h
CP C
JP NZ, et2
LD A, 01h
OUT (10h), A
JP et4
et2: LD A, 03h
CP C
JP NZ, et3
LD A, 00h
OUT (10h), A
JP et4
et3: LD A, 05h
CP C
JP NZ, et4
LD C, 00h
et4: INC C
EI
RETI
.END
```

c. Un microsistem cu microprocesor Z-80 gestionează șase celule de afișare de tip LED, cu anod comun, prin multiplexare, folosind un circuit Z-80 PIO. Schema de conectare este prezentată în figura următoare. Circuitul Z-80 PIO se află plasat în spațiul adreselor I/O la adresa de bază 60h cu C/D conectat la A₀ și la B/A la A₁. În același microsistem se mai află și un circuit Z-80 CTC la adresa de bază 70h. Ceasul procesorului are frecvența 3 MHz. Realizați un program prin intermediul căruia pe Led-uri se afișează ora exactă (hhmmss).



Soluție

În principiu, procesorul trebuie să rezolve în această problemă mai multe activități:

- afișarea prin multiplexare pe cele șase celule de afișare a informației de tip oră exactă;
- măsurarea timpului în vederea stabilirii etalonului de 1s necesar incrementării orei exacte;
- formarea informației ce trebuie afișată pe cele șase celule de afișare.

Afișarea prin multiplexare prezintă avantajul folosirii unui număr mic de circuite externe (decodificatoare binare - cod 7 segmente, circuite de tip latch). În caz general, în această tehnică, informația ce trebuie afișată se aplică ritmic fiecărei celule de afișare, frecvența de aprindere trebuind să fie mai mare decât frecvența de inerție a ochiului uman (50 Hz). La un moment dat doar o singură celulă este aprinsă, iar celelalte stinse. Bineînțeles, pentru a ajunge la aceeași luminanță ca și în situația afișării continue este necesară mărirea intensității curentului ce străbate LED-urile celulei de afișare.

Schema de afișare propusă este organizată într-o matrice de două rânduri și trei coloane. Simultan se vor aprinde două celule de afișare aflate pe aceeași coloană. Celulele de afișare sunt de tip anod comun, iar cele aflate pe aceeași coloană sunt comandate de același tranzistor de tip pnp (T21, T22, T23). Procesorul va trebui să dea comanda în vederea saturării pe rând a acestor tranzistori (în timp ce unul este saturat,

ceilalți doi sunt blocați). De aici rezultă, în condițiile schemei propuse, o frecvență de aprindere a coloanelor matricii de $3 \times 50\text{Hz} = 150\text{Hz}$ (minim). Dacă această frecvență este mai mică, senzația pentru un observator este de pâlpâire, ochiul uman neputând să realizeze integrarea. Dacă frecvența este mai mare, totul este normal, totuși această frecvență nu poate fi foarte mare datorită comutației imperfecte a dispozitivelor utilizate. Astfel, se poate ajunge în situația în care, dacă frecvența de aprindere este foarte mare, să nu existe timpii necesari iluminării celulelor. Vom alege în cadrul problemei noastre, lucrul la frecvența de 200 Hz (adică coloanele se aprind una după altă la 5 ms), astfel încât fiecare coloană se aprinde cu o frecvență de 66,5 Hz (sau în timp, după o perioadă de 15 ms). Fiecare coloană va rămâne aprinsă timp de 5 ms, timp suficient pentru ca LED-urile să se ilumineze. Pentru a avea o luminozitate suficientă, curentul prin fiecare LED va trebui mărit. Cum perioada de aprindere reprezintă a treia parte din perioada totală, vom mări curentul de trei ori față de situația aprinderii continue. La aprindere continuă, acest curent este de aproximativ 10 mA, și deci la aprinderea multiplexată vom avea un curent repetitiv de 30 mA (rămânând în limitele admise în foaia de catalog pentru LED-uri). Rezistențele de limitare a curentului prin LED-uri trebuie alese corespunzător.

În cadrul schemei propuse, comanda de aprindere-stingere a coloanelor se oferă prin liniile portului B (PB0, PB1, PB2) de la circuitul Z-80 PIO și al grupului de două tranzistoare (nnp-pnp). Un semnal "1"- logic va satura tranzistoarele permițând iluminarea, iar un "0" logic le va bloca, stingând coloana respectivă. Informația ce trebuie efectiv afișată se oferă prin liniile portului A de la circuitul Z-80 PIO, folosind circuitele de tip CMOS 4543 (decodificatoare binar - cod 7 segmente). La ieșirile acestor circuite sunt utilizați tranzistori pentru amplificarea curentului. Circuitele 4543 dispun de latch-uri interne pentru memorarea informației ce se aplică pe intrările lor (A, B, C, D). Memorarea informației se realizează prin folosirea intrării LD (load). Astfel, cât timp semnalul LD este "1" logic, latch-ul este transparent, adică informația de la intrări se transmite prin circuit către ieșiri. La trecerea din "1" logic în "0" logic a semnalului LD, latch-ul trece la memorarea informației aplicate la intrări. Cât timp LD este "0" logic se menține starea de memorare. Intrările de tip BL (blinking) și PH (Phase) ale circuitului 4543 sunt conectate la masă (recomandăm consultarea foii de catalog a circuitului pentru înțelegerea rolului acestora). Circuitele 4543 dispun de câte 7 ieșiri, câte una pentru fiecare segment comandat (a, b, c, d, e, f, g). Pentru schema noastră, un "1" logic la o astfel de ieșire va provoca aprinderea segmentului corespunzător din celula de afișare acționată, care primește +5V în anodul comun. De fapt, intervine în logică și tranzistorul T3 care inversează faza semnalului ce se aplică propriu-zis catodului diodei LED din celula de afișare. Circuitul 4543 este astfel realizat încât pentru orice combinație binară (0, ..., 9) aplicată la intrări, oferă la ieșiri codul 7 segmente corespunzător afișării. Orice combinație binară ce depășește 9 va genera la ieșirile circuitului 4543, "0" logic (LED-urile urmând să fie stinse).

În cadrul problemei noastre vom utiliza un canal al circuitului CTC pentru a genera o întrerupere la fiecare milisecundă (cu frecvența 1KHz). Întreruperile generate vor fi contabilizate de către procesor, astfel încât după cinci semnalizări (5 ms) se va schimba coloana ce va fi aprinsă la afișarea multiplexată. De asemenea, aceiași referință de timp (1 ms) se va folosi pentru obținerea intervalului de timp de 1 s necesar schimbării orei exacte.

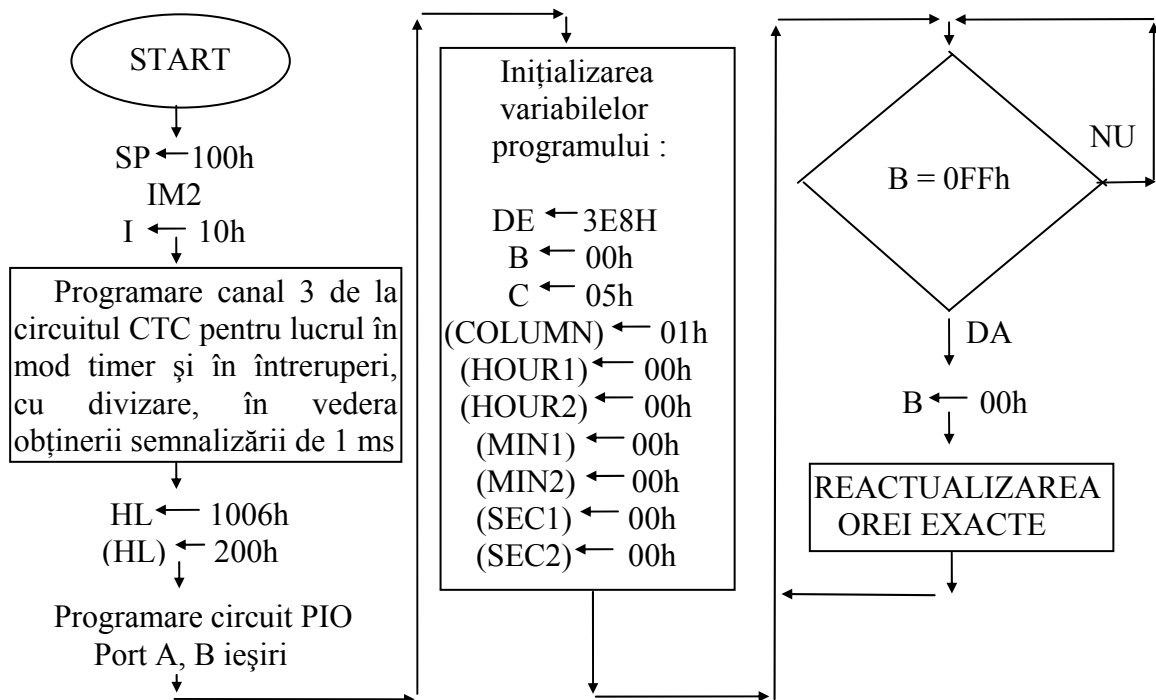
Programul va avea două componente:

- un program principal care va primi semnalizările de 1 secundă, caz în care va realiza reactualizarea orei exacte ce va fi oferită spre afișare;
- o subrutină de tratare a întreruperilor de la circuitul CTC (ce apare cu frecvența de 1 KHz) și care contorizează numărul acestor semnalizări. O primă sarcină a acestei subrutine este aceea de a gestiona afișarea multiplexată. Astfel, la 5 ms se va trece la reactualizarea coloanei ce se iluminează. Gestionarea afișării multiplexate va fi realizată tot în subrutina de tratare a întreruperii de la circuitul CTC. O a doua sarcină a subrutinei constă în stabilirea intervalului de timp de 1 s (trecerea a 1000ms), semnalizare ce se va face către programul principal, în vederea reactualizării orei exacte.

Vom folosi registrul DE al microprocesorului pentru contorizarea celor 1000 de milisecunde (3E8h), în vederea determinării intervalului de 1s. Registrul B va fi folosit pentru semnalizarea către programul principal a scurgerii intervalului de 1s. El va fi folosit pe post de semafor activ "1" logic. Vom folosi registrul C de la microprocesor pentru contorizarea și semnalizarea scurgerii intervalului de 5ms, după care are loc schimbarea coloanei ce se iluminează la afișarea multiplexată. Vom mai utiliza pentru afișarea multiplexată, încă 7 locații de memorie RAM din cadrul memoriei externe a microprocesorului. Prima locație va purta denumirea COLUMN și va fi folosită pentru a reține coloana ce se va aprinde de către programul principal. Această locație va putea avea trei valori: 01h, 02h și 03h. Următoarele 6 locații vor reține valoarea orei exacte și vor purta denumirile: HOUR1, HOUR2, MIN1, MIN2, SEC1, SEC2. Locațiile de tip 1 vor reține zecile, iar locațiile notate cu 2 vor memora unitățile elementelor de măsurare a timpului. Denumirile locațiilor se substituie practic adreselor acestora, echivalența fiind declarată la începutul programului sursă prin directiva EQU (equate) a asamblorului. Reactualizarea conținutului acestor locații se face de către programul principal la semnalizarea B=0FFh, semnalizare ce marchează trecerea intervalului de timp de 1 s. Conținutul respectivelor locații va fi trimis către celulele de afișare în cadrul gestionării afișării multiplexate, din subrutina ce tratează întreruperea de la circuitul CTC, în conformitate cu coloana ce se va ilumina la momentul respectiv. Practic, această parte de program lasă nemodificate conținuturile respectivelor locații de memorie.

Plasăm tabela vectorilor de întrerupere la adresa 1000h, iar subrutina de tratare a întreruperii de la circuitul CTC (canalul 3) la adresa 200h.

Organigrama generală a programului este:



Pentru programarea circuitului CTC, în vederea generării semnalizării de 1 secundă, vom folosi modul timer, cu divizarea semnalului de ceas de 3 Mhz. Utilizăm factorul de prescalare 16. De aici rezultă valoarea constantei de divizare prin operația:

$$\frac{3000KHz}{16 \cdot 1KHz} = 187,5$$

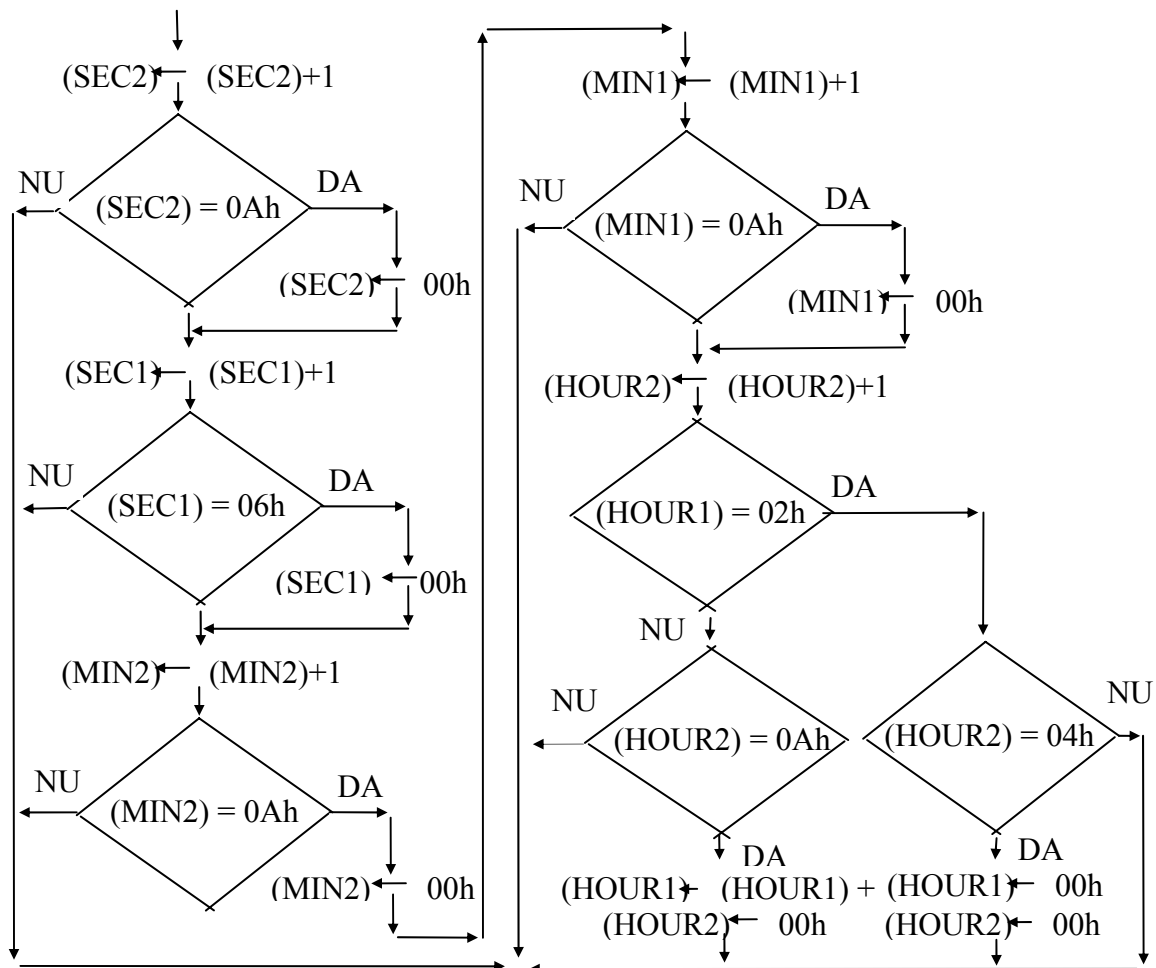
Considerăm constanta de divizare 187 (zecimal) sau 0BBh. De aici, următoarea secvență de programare a canalului 3 de la circuitul CTC:

(73h) ← 100001111 sau (73h) ← (87h) ; programarea registrului cuvântului de comandă al canalului 3 de la circuitul CTC, aflat la adresa 73h cu întreruperi, în modul timer, factor de prescalare 16, declanșare internă, automată, cu urmarea constantei de divizare și inițializare internă.

(73h) ← 0BBh ; constanta de divizare 187 la canalul 3 de la CTC

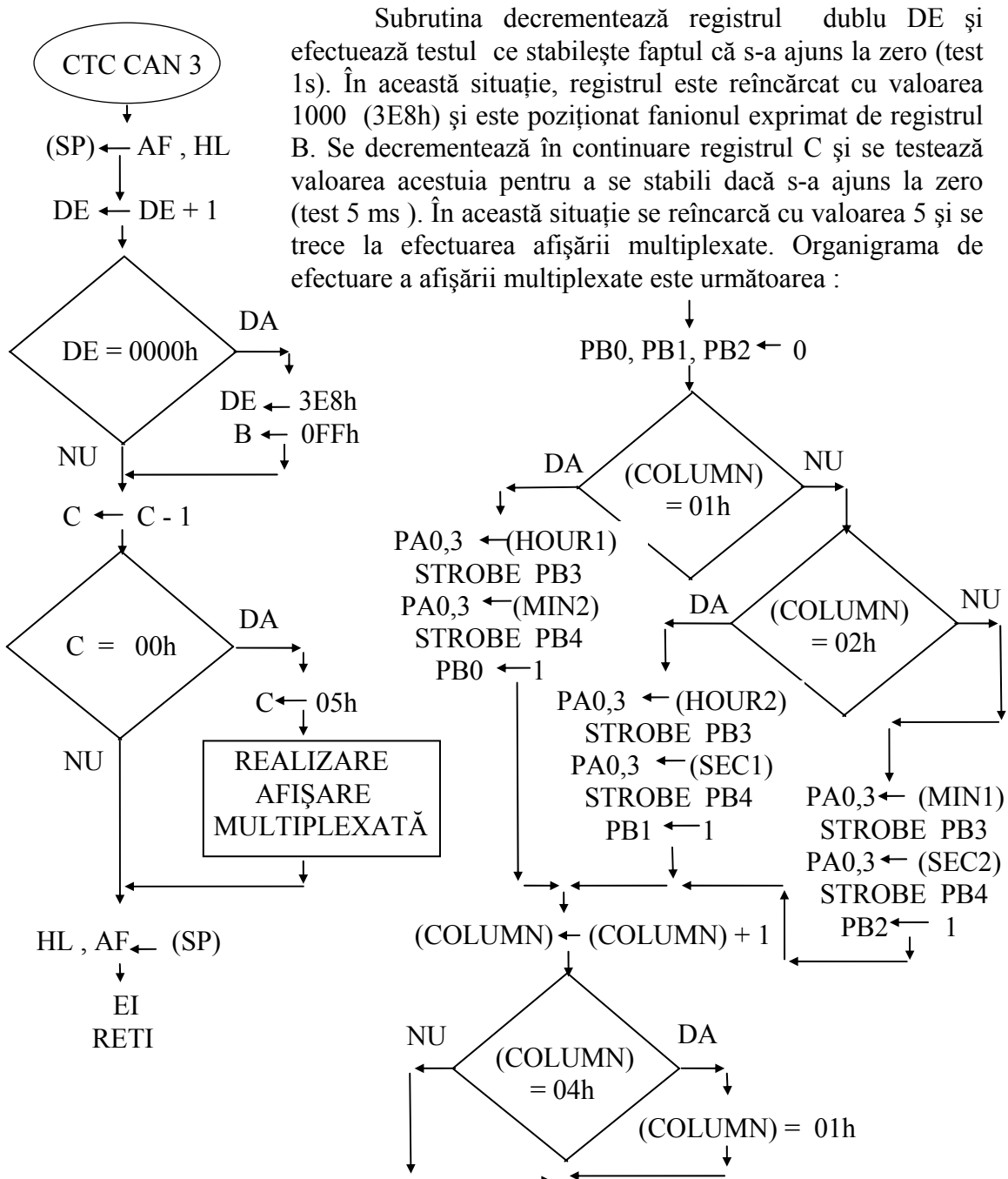
(70h) ← 00h; vectorul de întrerupere al circuitului CTC, trimis pe adresa canalului 0.

Reactualizarea orei exacte se face conform următoarei organigrame:



Reamintim că notația cu paranteze (xx) reprezintă conținutul locației de memorie ce are adresa xx. Organigrama începe cu incrementarea unităților de secunde și testul ce stabilește dacă a fost depășită valoarea 9. Dacă nu s-a depășit, reactualizarea s-a terminat. Astfel, unitățile de secunde sunt aduse la zero și se incrementează zecile de secunde. Aici testul se face pentru a stabili dacă a fost depășită valoarea 5. Continuarea se face într-un mod similar trecând prin unități de minute, zeci de minute, ore și zeci de ore. Organigrama prezintă detaliază blocul logic al reactualizării orei exacte din organigrama generală a programului principal.

Organigrama subrutinei de tratare a întreruperii generate de canalul 3 al circuitului CTC este următoarea:



În conformitate cu structura hardware a afisajului multiplexat s-au făcut următoarele alocări pentru celulele de afișare :

- prima coloană celula 1 HOUR1 (zeci de ore)
 celula 4 MIN2 (unități de minute)
- coloana a doua celula 2 HOUR2 (unități de ore)
 celula 5 SEC1 (zeci de secunde)
- coloana a treia celula 3 MIN1 (zeci de minute)
 celula 6 SEC2 (unități de secunde)

În organigramă apar precizate și semnale de tip strobe ce se aplică prin liniile PB3 și PB4 de la circuitul Z-80 PIO la intrările de tip LD (load) ale circuitelor 4543. Un astfel de semnal va provoca încărcarea informațiilor generate pe ieșirile portului A PA0,.....,PA3 în latch-urile circuitelor 4543. Strobe-ul va fi generat prin ridicarea semnalului în “1” -logic, urmat de trecerea acestuia în “0” - logic.

În esență, acțiunile ce se realizează pentru afișarea multiplexată sunt stingerea inițială a tuturor celulelor de afișare (prin trecerea liniilor PB0, PB1, PB2 în “0” - logic), urmată de testul ce indică coloana care va fi iluminată. Funcție de numărul acestei coloane se aplică informația corespunzătoare pentru celulele de afișare respective, folosind și semnalul de tip strobe, ce provoacă memorarea datelor în circuitele 4543, după cum s-a menționat mai sus. După efectuarea memorărilor, se iluminează coloana prin trecerea liniei de tip PB corespunzătoare (PB0, PB1, PB2) în “1” - logic. Stingerea inițială a afișajului, memorarea informațiilor și aprinderea noii coloane conduc la o afișare corectă, care apare continuă privirii umane, fără “resturi” de la o coloană la alta. În final, are loc incrementarea variabilei ce memorează coloana următoare, care va fi iluminată. În caz că se depășește valoarea 03h, această variabilă se reîncarcă cu valoarea 01h (numărul primei coloane).

În limbaj de asamblare obținem pentru programul principal :

```

                .ORG 100h
;
COLUMN        EQU 2000h
HOUR1          EQU 2001h
HOUR2          EQU 2002h
MIN1           EQU 2003h
MIN2           EQU 2004h
SEC1           EQU 2005h
;
                LD SP, 100h
                IM2
                LD A, 10h
                LD I, A
                LD A, 87h
                OUT (73h), A
                LD A, 0BBh
                OUT (73h), A
                LD A, 00h
                OUT (70h), A
                LD HL, 1006h

                LD A, 00h
                LD (HOUR2), A
                LD A, 00h
                LD (MIN1), A
                LD A, 00h
                LD (MIN2), A
;
                LD (HL), 200h
                LD A, 0Fh
                OUT (61h), A
                OUT (63h), A
                LD DE, 3E8h
                LD B, 00h
                LD C, 05h
                LD A, 01h
                LD (COLUMN), A
                LD A, 00h
                LD (HOUR1), A

                CP 06h
                JP NZ, ett1
                LD A, 00h
                LD (MIN2), A

                LD HL, MIN1
    
```

	LD A, 00h		INC (HL)
	LD (SEC1), A		LD A, (MIN1)
	LD A, 00h		CP 06h
	LD (SEC2), A		JP NZ, ett1
;			LD A, 00h
ett1:	LD A, B		LD (MIN1), A
	CP 0FFh	;	
	JP NZ, ett1		LD HL, HOUR2
	LD B, 00h		INC (HL)
;			LD A, (HOUR1)
react:	LD HL, SEC2		CP 02h
	INC (HL)		JP Z, ett2
	LD A, (SEC2)		LD A, (HOUR2)
	CP 0Ah		CP 0Ah
	JP NZ, ett1		JP NZ, ett1
	LD A, 00h		LD HL, HOUR1
	LD (SEC2), A		INC (HL)
;			LD A, 00h
	LD HL, SEC1		LD (HOUR2), A
	INC (HL)		JP ett1
	LD A, (SEC1)	ett2:	LD A, (HOUR2)
	CP 06h		CP 04h
	JP NZ, ett1		JP NZ, ett1
	LD A, 00h		LD A, 00h
	LD (SEC), A		LD (HOUR1), A
;			LD (HOUR2), A
	LD HL, MIN2		JP ett1
	INC (HL)	;	
	LD A, (MIN2)		

Pentru subrutina de tratare a întreruperii generate de canalul trei al circuitului CTC, obținem în limbaj de asamblare următoarea formă sursă:

. ORG 200h		LD B, 0FFh
PUSH AF	sett1:	DEC C
PUSH HL		JP NZ, sett2
DEC DE	;	
LD A, E		LD C, 05h
JP NZ, sett1		LD A, 00h
LD A, D		OUT (72h), A
JP NZ, sett1	;	
LD DE, 3E8h		LD A, (COLUMN)
CP 01h		LD A, (SEC2)
JP NZ, sett3		OUT (70h), A
;		LD A, 10h
		OUT (72h), A
LD A, (HOUR1)		LD A, 00h
OUT (70h), A		OUT (72h), A
LD A, 08h		

```

OUT (72h), A      ;
LD A, 00h          ;
OUT (72h), A      ;
LD A, (MIN2)      ;
OUT (70h), A      ;
LD A, 10h          ;
OUT (72h), A      ;
LD A, 00h          ;
OUT (72h), A      ;
LD A, 01h          ;
OUT (72h), A      ;
JP sett4          ;
LD A, (COLUMN)    ;
CP 02h            ;
JP NZ, sett5      ;
LD A, (HOUR2)     ;
OUT (70h), A      ;
LD A, 08h          ;
OUT (72h), A      ;
LD A, 00h          ;
OUT (72h), A      ;
LD A, (SEC1)      ;
OUT (70h), A      ;
LD A, 10h          ;
OUT (72h), A      ;
LD A, 00h          ;
OUT (72h), A      ;
LD A, 01h          ;
OUT (72h), A      ;
JP sett4          ;
LD A, (MIN1)      ;
OUT (70h), A      ;
LD A, 08h          ;
OUT (72h), A      ;
LD A, 00h          ;
OUT (72h), A      ;
LD HL, COLUMN
INC (HL)
LD A, (COLUMN)
CP 04h
JP NZ, sett2
LD A, 01h
LD (COLUMN), A
POP HL
POP AF
EI
RETI
.END

```

b. (Aplicație practică) În spațiul IO al unui microsistem cu microprocesor Z80 se află un circuit Z80 PIO la adresa de bază 00h, cu conexiunile B/A – A1 și C/D – A0 și un circuit Z80 CTC la adresa de bază 20h cu CS1 – A1 și CS0 – A0. Pe portul A al circuitul PIO se află următoarele conexiuni: PA0 – LED ROSU, PA1 – LED GALBEN, PA2 – BUTON, PA3 – DIFUZOR. Pe portul B se află conectată o matrice de LED-uri 7x5 conform schemei de mai jos:

PB7	PB6	PB5	PB4	PB3	PB2	PB1	PB0
Selecție linie-coloana	Data 6	Data 5	Data 4	Data 3	Data 2	Data 1	Data 0

Selecția linie-coloană funcționează astfel: pe frontul crescător este transmisă informația de coloană iar pe cel descrescător informația de linie. Un LED este aprins dacă pe linie și pe coloana pe care se află se aplică 1 logic.

Să se realizeze un program care va afișa continuu cifra 5.

Rezolvare:

Se va folosi metoda de afișare multiplexată, pentru a reduce consumul. Astfel la un moment dat se va aprinde o singură coloană. Pentru ca cifra să fie percepută continuu frecvența de afișare a ei trebuie să fie de 50Hz. Frecvența de afișare a unei coloane va fi deci de $50 \times 5 = 250\text{Hz}$.

Se va folosi pentru generarea temporizării circuitul Z80 CTC. O posibilitate de a obține frecvența de 250 Hz este următoarea:

$$5\,000\,000 / 16 = 312\,500 / 50 = 6250 / 25 = 250.$$

Pentru obținerea unui semnal cu frecvența de 250Hz se va programa astfel:

canal 0:

temporizator, prescaler 16, constata divizare 50, fără întrerupere.

cuvântul de comandă este:

$$0000\,0111\,(2) = 07h.$$

canal 1:

numărător, constata divizare 25, cu întrerupere.

cuvântul de comandă este:

$$1100\,0111\,(2) = C7h.$$

Pentru circuitul Z80 PIO se va folosi portul A în modul 3 de lucru (ca la problema anterioară) și portul B în modul 0:

Port A:

mod 3.

cuvântul de mod:

$$1100\,1111\,(2) = CFh$$

cuvântul de direcție:

$$1111\,0100\,(2) = F4h.$$

Port B:

mod 0.

cuvântul de mod:

$$0000\,1111 = 0Fh$$

Imaginea cifrei este salvată în memorie, pe 5 octeți (unul pentru fiecare coloană) și a fost realizată după construcția următoarei matrice:

```
11111
10000
10000
11111
```

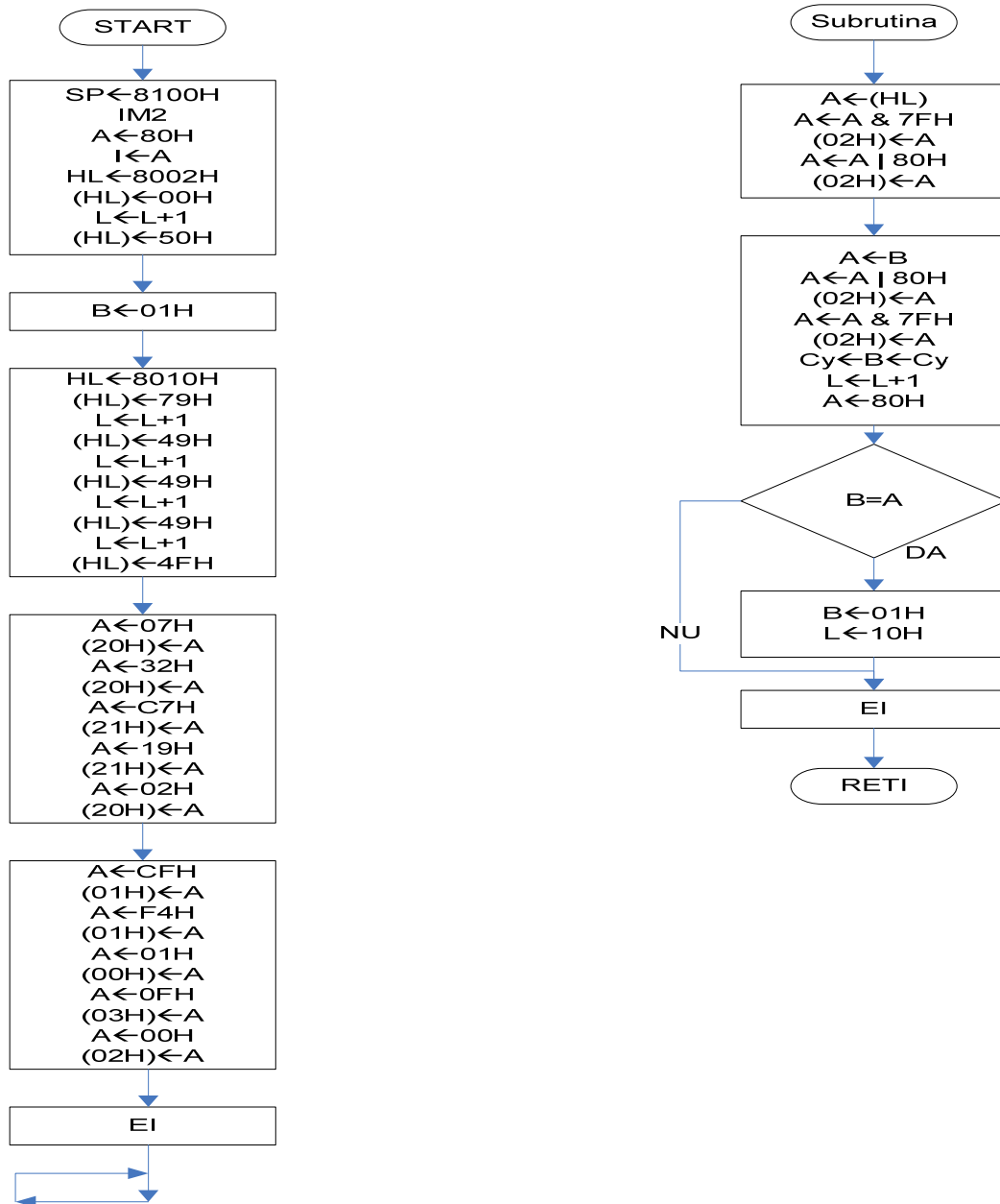
00001

00001

11111

Subrutina de tratare a intreruperii se va plasa la adresa 5000h iar tabela vectorilor de intrerupere la 8000h. Scrierea în tabela de intrerupere se face în program folosind registrul HL. Stiva este initializată la adresa 8100h.

Registrii de uz general folositi: HL pentru preluarea imaginii cifrei din memorie,
B numărator de coloane (indică coloana curentă activată).



org 4000h

;ceas 5 000 000 / 16 = 312 500 / 50 = 6250 / 25 = 250 frecventa de baleiaj coloane

;cuv comanda ch0: 0000 0111 = 07 temporizator, prescaler 16, constanta 50

;cuv comanda ch1: 1100 0111 = C7 numarator, constanta 25

;pio modul 3 portul a 1100 1111 = cf directie 1111 0100 = f4

; modul 0 portul b 0000 1111 = 0f

;=====INITIALIZARI=====

;initializare cpu

ld sp,8100h ;initializare stiva

im 2 ;activare mod intreruperi 2

ld a,80h ;initializare registru I

ld i,a

ld hl,8002h ;initializare tabela vectori intrerupere

ld (hl),00h

inc l

ld (hl),50h

;initializare variabile

ld b,01h ;numarator coloane

;;;;;;;;;initializare spatiu de memorie cu informatia de linii

; 11111

; 10000

; 10000

; 11111

; 00001

; 00001

; 11111

ld hl,8010h ;initializare imagine cifra

ld (hl),79h

inc l

ld (hl),49h

inc l

ld (hl),49h

inc l

ld (hl),49h

inc l

ld (hl),4fh

;programare ctc

ld a,07h ;canal 0

out (20h),a

ld a,032h

out (20h),a

ld a,0C7h ;canal 1

out (21h),a

ld a,019h

out (21h),a

ld a,02h ;vector intrerupere

```

out (20h),a
;programare pio
ld a,0cfh          ;port A
out (01h),a
ld a,0f4h
out (01h),a
ld a,01h
out (00h),a

ld a,0fh          ;port B
out (03h),a
ld a,00h
out (02h),a
ei                ;activare intreruperi
buclamain:        ;bucă infinita
jp buclamain

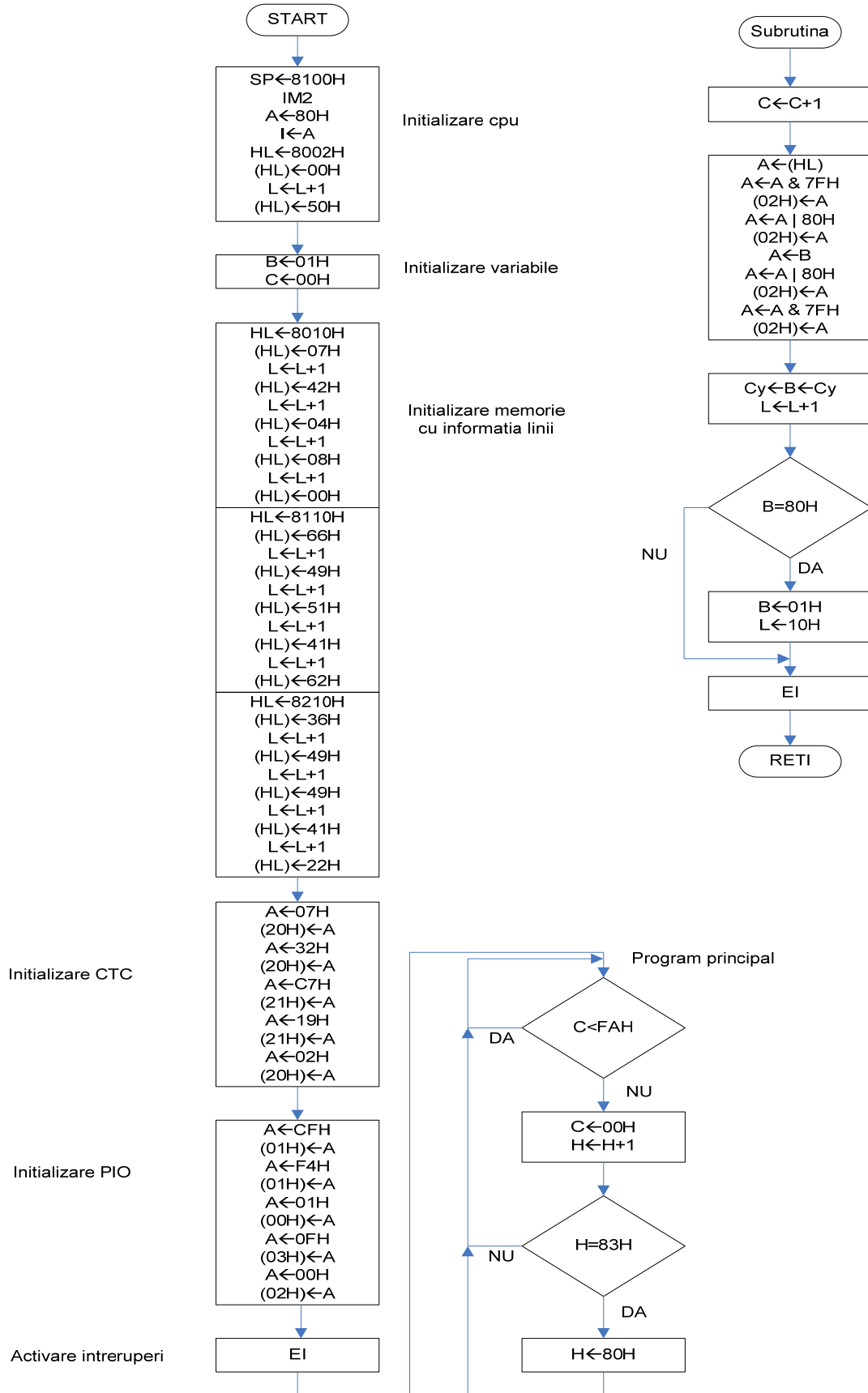
org 5000h          ;subrutina tratare intrerupere
ld a,(hl)         ;depunere informatie coloana
and 7fh
out (02h),a
or 80h
out (02h),a

ld a,b            ;depunere informație linie
or 80h
out (02h),a
and 7fh
out (02h),a
rl b              ;modificare coloană activata
inc l
ld a,80h
cp b
jp nz,et0i        ;test maxim coloana
ld b,01h
ld l,10h
et0i:
ei
reti
end

```

Probleme

1. Să se schimbe forma cifrei (literei) afișate.
- d) (Aplicație practică) În contextul enunțat la problema de la punctul anterior să se realizeze un program care să afișeze, pe matricea de LED-uri, cu o frecvență de o secundă, consecutiv cifrele 1, 2 și 3.



Rezolvare

Inițializarea hardware se păstrează similar cu problema anterioară. Datorită faptului că subrutina de întrerupere apare cu o frecvență de 250Hz sau cu o perioadă de 4ms se va folosi un contor de 250 pentru a realiza o temporizare de 1 secundă. De asemenea vor fi generate imagini pentru toate cele trei cifre care vor fi plasate în memorie la adresele: 8010h – cifra 1, 8020h – cifra 2 și 8030h – cifra 3.

Setările hardware sunt similare cu problema anterioară.

Registrii de uz general folosiți: HL pentru preluarea imaginii cifrei din memorie,
B numărător de coloane (indică coloana curentă activată),
C contor temporizare 1 secundă.

```
org 4000h
;ceas 5 000 000 / 16 = 312 500 / 50 = 6250 / 25 = 250 frecventa de baleiaj coloane
;cuv comanda ch0: 0000 0111 = 07 temporizator, prescaler 16, constanta 50
;cuv comanda ch1: 1100 0111 = C7 numarator, constanta 25
;pio modul 3 portul a 1100 1111 = cf directie 1111 0100 = f4
; modul 0 portul b 0000 1111 = 0f
;=====INITIALIZARI=====
;initializare cpu
ld sp,8100h      ;initializare stivă
im 2             ;mod 2 tratare intreruperi
ld a,80h         ;initializare registru I
ld i,a
ld hl,8002h      ;initializare tabela vectori intrerupere
ld (hl),00h
inc l
ld (hl),50h

;initializare variabile
ld c,00h         ;numarator 4 milisecunde (pentru temporizare software 1 secunda)
ld b,01h         ;numarator coloane
;,,,,,initializare spatiu de memorie cu informatia de linii - cifra 1
; 00001
; 00011
; 00101
; 01001
; 00001
; 00001
; 00011
ld hl,8010h
ld (hl),07fh
inc l
ld (hl),42h
inc l
```

```
ld (hl),04h
inc l
ld (hl),08h
inc l
ld (hl),00h
,,,,,,initializare spatiu de memorie cu informatia de linii - cifra 2
; 01110
; 10001
; 00001
; 00010
; 00100
; 10001
; 11111
ld hl,8110h
ld (hl),066h
inc l
ld (hl),49h
inc l
ld (hl),51h
inc l
ld (hl),41h
inc l
ld (hl),62h
,,,,,,initializare spatiu de memorie cu informatia de linii - cifra 3
; 01110
; 10001
; 00001
; 00110
; 00001
; 10001
; 01110
ld hl,8210h
ld (hl),036h
inc l
ld (hl),49h
inc l
ld (hl),49h
inc l
ld (hl),41h
inc l
ld (hl),22h

;programare etc
ld a,07h                ;canal 0
out (20h),a
ld a,032h
```

```

out (20h),a

ld a,0C7h                ;canal 1
out (21h),a
ld a,019h
out (21h),a

ld a,02h                ;vector intrerupere
out (20h),a

;programare pio
ld a,0cfh                ;port A
out (01h),a
ld a,0f4h
out (01h),a
ld a,01h
out (00h),a

ld a,0fh                ;port B
out (03h),a
ld a,00h
out (02h),a

ei                        ;activare intrerupere

;=====PROGRAM PRINCIPAL=====
buclamain:                ;program principal
ld a,0fah                ;testare contor secunda
cp c
jp nc,buclamain
ld c,00h                ;resetare daca a depasit valoarea de 250
inc h                    ;incrementare adresa cifra afisata
ld a,83h
cp h
jp nz,buclamain          ;testare adresa maxima
ld h,80h
jp buclamain

;subrutina timer: 250 Herti sau 4 milisecunde
org 5000h                ;subrutina tratare intrerupere

inc c                    ;incrementare contor secunda

ld a,(hl)                ;depunere informație coloana
and 7fh

```



```

out (02h),a
or 80h
out (02h),a
ld a,b                ;depunere informație linie
or 80h
out (02h),a
and 7fh
out (02h),a

rl b                  ;modificare coloană activată
inc l
ld a,80h
cp b
jp nz,et0i           ;test coloana maxima
ld b,01h
ld l,10h
et0i:
ei
reti

end

```

4. DESFĂȘURAREA LUCRĂRII

1. Se va citi și conspecta breviarul teoretic. Se atrage atenția asupra faptului că toate cunoștințele căpătate în această lucrare vor fi necesare și în derularea celorlalte lucrări.
2. Se vor studia problemele rezolvate, încercând găsirea altor posibilități de rezolvare.
3. Programele obiect ale problemelor rezolvate vor fi introduse în memoria microsistemelor existente în laborator (prin intermediul programului monitor) și se vor rula. Se va observa efectul rulării și buna lor execuție.
4. Se vor rezolva următoarele probleme propuse și se va urmări execuția corectă prin introducerea programelor obiect în memoria microsistemelor și rulare:
 - a) Într-un microsistem cu microprocesor Z-80 se află conectate în spațiul adreselor I/O un circuit Z-80 PIO și, respectiv, un circuit Z-80 CTC. Frecvența semnalului de ceas ce se aplică procesorului este de 2,5 MHz. Pe linia PA0 de la circuitul PIO se aplică un semnal extern oscilatoriu TTL cu frecvența necunoscută, dar mai mică de 1KHz. Realizați un program pentru determinarea frecvenței necunoscute plecând de la definiția frecvenței unui semnal (număr de impulsuri în perioada de timp - 1 secundă). Circuitul CTC va fi folosit pentru determinarea perioadei de 1 secundă, iar rezultatul va fi plasat în registrul DE al procesorului. Adresa circuitului PIO este 30h (cu C/D și B/A conectate la A0 și, respectiv, A1), iar a circuitului CTC este 40h. Nu se vor folosi construcții hardware suplimentare.

b) În același microsistem ca și în problema anterioară, pe linia PB0 a circuitului PIO, prin intermediul unui amplificator se află conectat un difuzor. Realizați un program prin intermediul căruia către difuzor se emit semnale cu frecvențele de 1KHz, 0.5 KHz și 0.25 KHz fiecare având durată de 1s, repetat. Nu se vor utiliza construcții hardware suplimentare și nici alte semnale oscilatorii care vor proveni din exteriorul sistemului.

5. Răspundeți la următoarele întrebări :

a) Pentru prima problemă rezolvată indicați ce modificări ar trebui să fie făcute pentru ca factorul de umplere al semnalului generat să fie 4/5; prezentați organigrama programului în acest caz.

b) Pentru prima problemă rezolvată indicați de ce tabela vectorilor de întrerupere este scrisă începând cu adresa 0206h.

c) Câte locații de stivă folosește prima problemă rezolvată și care sunt acestea ?

d) Ce modificare ar trebui adusă în cea de-a doua problemă rezolvată pentru ca tipărirea cifrei "0" să se facă la intervale de 3 secunde ?

e) Care este numărul maxim de locații de stivă folosit în cea de-a doua problemă rezolvată?

f) De ce este necesară testarea lui TxRDY în cadrul celei de-a două probleme rezolvate și cum se face acest lucru ?

g) Pentru afișajul prezentat la cea de-a treia problemă rezolvată, în condițiile în care Vcc este +5V, ce element al schemei trebuie dimensionat și cum, pentru ca prin LED-uri, curentul repetitiv ce se stabilește să fie 30 mA (prezentați calculul respectiv) ?

h) Ce rol au rezistențele R21, R22, R23 în schemă ? Pot să nu existe ?

i) Pentru cea de-a treia problemă rezolvată, la subrutina de tratare a întreruperii, cum explicați necesitatea salvării în stivă a regiștrilor AF și HL ?

j) Câte locații de stivă sunt folosite de către programul celei de-a treia probleme rezolvate?

k) Ce modificări ar trebui aduse programului pentru ca ora cea mai mare care poate fi afișată să fie 12:59:59 ? Care este ora maximă afișată de programul prezentat în lucrare ?